

Rivanna — run your Jupyter notebook as a Slurm job (Beau)

Bundle: folder `Beau_guide/` — unzip it on Rivanna and `cd` there; `./pipe_job-beau.slurm` means “this folder.”

Audience: Anyone moving from “notebook on the login node” to “notebook on a compute node,” with optional skip flags and live logs.

How to read this

- Normal paragraphs = do these steps in order.
- Blockquotes (`>`) = optional background — skip anytime.

The picture

Where	Good for
Login node	Editing notebooks, short tests, <code>sbatch</code> , <code>tail</code> , checking queue
Compute node (Slurm job)	Anything that runs a long time or needs reserved CPUs/RAM

Slurm runs your notebook **non-interactively**: one pass **top to bottom**, same as “Run All,” then saves an **output** `.ipynb` with results baked in.

Why not interactive Jupyter on a compute node?

You can use Open OnDemand Jupyter on Rivanna too — this guide is for “batch execute my notebook like a script,” which is simple to automate and matches how heavy pipelines are usually run.

Step 1 — Notebook pattern (optional): booleans in early cells

Put switches in **Cell 0** (or the first few code cells): paths, options, and booleans such as `RUN_DOWNLOAD = True`, `RUN_FIT = False`.

Later cells wrap heavy work:

```
if RUN_FIT:
    # expensive step
    ...
else:
    # load cached results
    ...
```

Rename freely (`RUN_STEP2`, etc.). The batch script looks for lines starting with `RUN_CELL` only so it can **echo** them to the log before running — purely cosmetic; use `RUN_CELL3 = False` if you want that echo.

Why bother?

One notebook file, many modes: flip booleans instead of copying the notebook or commenting out huge blocks.

Step 2 — Environment

1. `module load miniforge` (same idea as inside `pipe_job_beau.slurm`).
2. A `conda env` whose name matches `ENV_NAME` in the Slurm script (default there is `tenure_net` — change it to yours).
3. In that env: `papermill` (recommended) or `jupyter` for `nbconvert --execute`.

Step 3 — Edit `pipe_job_beau.slurm`

Replace placeholders:

Placeholder	Meaning
<code>YOUR_NETID@virginia.edu</code>	Mail notices
<code>YOUR_SLURM_ALLOCATION</code>	Billing (<code>#SBATCH --account</code>)
<code>--partition</code> , <code>--mem</code> , <code>--cpus</code> , <code>--time</code>	Fit your job
<code>ENV_NAME</code>	Your conda env name
<code>NB_IN</code>	Notebook filename relative to the working directory (see Step 4), e.g. <code>analysis.ipynb</code> or <code>runs/main.ipynb</code>
<code>BEAU_PROJECT_ROOT</code> (optional)	Absolute path if you don't want to <code>cd</code> before <code>sbatch</code>

Step 4 — Submit

Working directory for papermill is chosen in this order:

1. `BEAU_PROJECT_ROOT` set inside the script (non-empty), or
2. `PROJECT_ROOT` exported in your shell before `sbatch` , or
3. The folder you `cd` into before `sbatch` (`SLURM_SUBMIT_DIR`).

Recommended: put `your_notebook.ipynb` inside `Beau_guide/`, set `NB_IN` accordingly, `cd` into `Beau_guide/`, then:

```
cd ~/Beau_guide # unpacked bundle (contains pipe_job_beau.slurm)
sbatch ./pipe_job_beau.slurm
```

Advanced — notebook lives elsewhere: `cd` to that folder before `sbatch`, or set `BEAU_PROJECT_ROOT` / `PROJECT_ROOT` so `NB_IN` still resolves.

You do **not** need Git — only a consistent working directory for `NB_IN`.

Log files (`slurm-<jobname>-<JOBID>.out` / `.err`) land **where you ran** `sbatch` (often `Beau_guide/` if you `cd` there).

Step 5 — Live feedback

Progress bars (`tqdm`) and much library output go to `stderr` → `.err` file.

From the directory where you submitted:

```
tail -f slurm-pipe_job_beau-<JOBID>.err
```

Or use the helper from `Beau_guide/` (or `export PROJECT_ROOT=` your submit directory):

```
chmod +x ./track_slurm_beau.sh # once
./track_slurm_beau.sh <JOBID>
```

Stdout (`.out`) often has papermill cell markers — useful too:

```
tail -f slurm-pipe_job_beau-<JOBID>.out
```

`tail -f` keeps streaming new lines. **Ctrl+C** stops `tail` only — not the Slurm job (`scancel JOBID` cancels the job).

Step 6 — Result file

After the job finishes:

`slurm-pipe_job_beau-<JOBID>-output.ipynb` — executed notebook **with outputs**, next to your input notebook (under the **working directory** from Step 4).

Step 7 — Cancel

```
scancel <JOBID>
```

Troubleshooting

Issue	What to check
<code>NB_IN</code> not found	<code>NB_IN</code> is relative to papermill working dir (Step 4). If you <code>sbatch</code> from <code>Beau_guide/</code> , either put the notebook there or override <code>BEAU_PROJECT_ROOT</code> .
Can't find <code>.err</code>	Logs are under <code>SLURM_SUBMIT_DIR</code> — <code>cd</code> to where you ran <code>sbatch</code> before <code>track_slurm_beau.sh</code> , or set <code>PROJECT_ROOT</code> .
<code>Permission denied</code> on tracker	<code>chmod +x ../track_slurm_beau.sh</code>

Optional: `.env` / secrets

If your notebook loads `.env`, keep keys out of Git — see `.env.example_beau` for the idea only.

Reference files in this folder

`*_reference_beau.*` — copies of Charles's Ivy_Net `pipe_job` / `track_slurm` style for comparison; **not required** for your workflow.